



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

**Институт информационных технологий (ИИТ)
Кафедра прикладной математики (ПМ)**

**ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №5
«КОРРЕЛЯЦИЯ И
РЕГРЕССИЯ»**

по дисциплине «Языки программирования для статистической обработки
данных»

Студент группы

ИНБО-20-23. Боргачев Т.М.

(подпись)

Преподаватель

Трушин СМ

(подпись)

Москва 2025 г.

ВВЕДЕНИЕ

Цель и задачи работы

Цель практики: освоить методы анализа корреляций и построения линейных регрессионных моделей с использованием Python, R.

Задачи практики:

1. Рассчитать корреляцию между переменными:

- Вычисление коэффициентов корреляции Пирсона и Спирмена.
- Построение и визуализация корреляционной матрицы:
- Python: pandas, seaborn.
- R: cor(), ggcorrplot.

2. Построить линейную регрессионную модель:

- Анализ зависимости одной переменной от другой.
- Интерпретация результатов регрессии (коэффициенты, R^2).
- Python: statsmodels, sklearn.
- R: lm().

3. Сравнить подходы к расчёту корреляций и регрессии в Python, R.

4. Выявить преимущества и ограничения каждого инструмента для анализа данных.

Краткое описание методов корреляции и регрессии.

Корреляция измеряет степень взаимосвязи между двумя переменными. Коэффициент корреляции Пирсона оценивает линейную зависимость, принимая значения от -1 до 1, где 0 означает отсутствие линейной связи. Коэффициент Спирмена оценивает монотонную зависимость и устойчив к выбросам.

Линейная регрессия моделирует зависимость одной переменной (зависимой) от другой (независимой) в виде уравнения $y=a+bx$, где b — коэффициент наклона, a — свободный член, а R^2 показывает долю дисперсии зависимой переменной, объясненную моделью.

РЕЗУЛЬТАТЫ РАБОТЫ

2.1. Построение корреляционных матриц

2.1.1. Python (pandas, seaborn)

Для анализа использовались данные о доходах и расходах. Код для расчета и визуализации корреляции представлен в Листинге 1.

Листинг 1– Расчет и визуализация корреляции в Python

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
df = pd.read_csv("mxmh_survey_results.csv")
df_numeric = df[["Age", "Hours per day",
"BPМ", "Anxiety", "Depression", "Insomnia", "OCD"]]
pearson_corr = df_numeric.corr(method='pearson')
spearman_corr = df_numeric.corr(method='spearman')
print("Корреляция Пирсона:\n", pearson_corr)
print("Корреляция Спирмена:\n", spearman_corr)

plt.figure(figsize=(8, 6))
sns.heatmap(pearson_corr, annot=True, cmap='coolwarm')
plt.title("Корреляционная матрица Пирсона")
plt.show()
```

Результаты представлены на Рисунке 1.



Рисунок 1 - Корреляционная матрица в Python (seaborn)

2.1.2. R (cor(), ggcorrplot)

Код для анализа корреляции в R представлен в Листинге 2.

Листинг 2 – Расчет и визуализация корреляции в R

```
library(readr)
library(ggcorrplot)
df <- read_csv("mxmh_survey_results.csv")
head(df)
library(dplyr)
numeric_df <- df %>% select(where(is.numeric))

pearson_corr <- cor(numeric_df, method = "pearson")
spearman_corr <- cor(numeric_df, method = "spearman")
print("Корреляция Пирсона:")
print(pearson_corr)
print("Корреляция Спирмена:")
print(spearman_corr)
ggcorrplot(spearman_corr, lab = TRUE, title = "Корреляционная матрица Спирмена")
```

Результаты представлены на Рисунке 2.

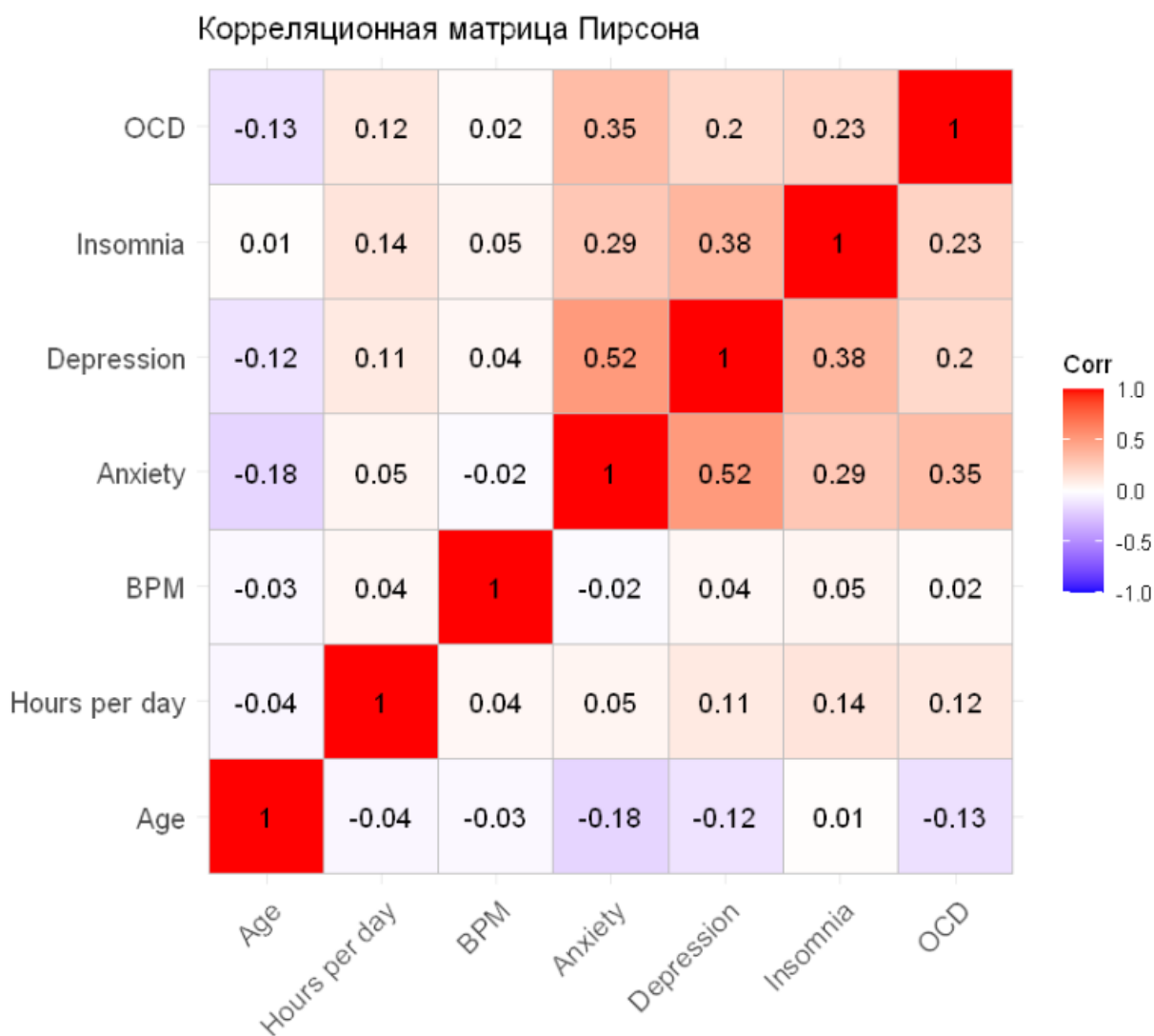


Рисунок 2 - Корреляционная матрица в R (ggcorrplot)

Линейная регрессия

Python (statsmodels)

Модель регрессии зависимости расходов от доходов построена с использованием statsmodels. Код представлен в Листинге 3.

Листинг 3 – Построение регрессии в Python

```
import statsmodels.api as sm
df["Depression"] = df["Depression"].fillna(df["Depression"].mean())
df["Age"] = df["Age"].fillna(df["Age"].mean())
# Предположим, что 'Independent_Variable' – столбец в df
# и 'Dependent_Variable' – целевая переменная
X = df['Age'].unique() # Независимая переменная
Y = df['Depression'].groupby(by=df['Age']).mean() # Зависимая переменная
X = sm.add_constant(X) # Добавляем константу
model = sm.OLS(Y, X).fit() # Строим модель
print(model.summary()) # Сводная статистика

plt.scatter(X[:, 1], Y)
plt.plot(X[:, 1], model.predict(X),
linewidth=2, color = 'red')
plt.title("Линейная регрессия")
plt.xlabel("Независимая переменная")
plt.ylabel("Зависимая переменная")
plt.show()
```

Результаты представлены на Рисунке 3.

OLS Regression Results						
=====						
Dep. Variable:	Depression		R-squared:	0.118		
Model:	OLS		Adj. R-squared:	0.104		
Method:	Least Squares		F-statistic:	8.058		
Date:	Tue, 08 Apr 2025		Prob (F-statistic):	0.00617		
Time:	15:16:03		Log-Likelihood:	-127.67		
No. Observations:	62		AIC:	259.3		
Df Residuals:	60		BIC:	263.6		
Df Model:	1					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

const	5.3286	0.577	9.235	0.000	4.174	6.483
x1	-0.0348	0.012	-2.839	0.006	-0.059	-0.010
=====						
Omnibus:	7.166	Durbin-Watson:	1.806			
Prob(Omnibus):	0.028	Jarque-Bera (JB):	10.877			
Skew:	0.279	Prob(JB):	0.00435			
Kurtosis:	4.975	Cond. No.	111.			
=====						

Рисунок 3 – Результаты линейной регрессии в Python

Визуализация модели на языке Python представлена на рис. 4.

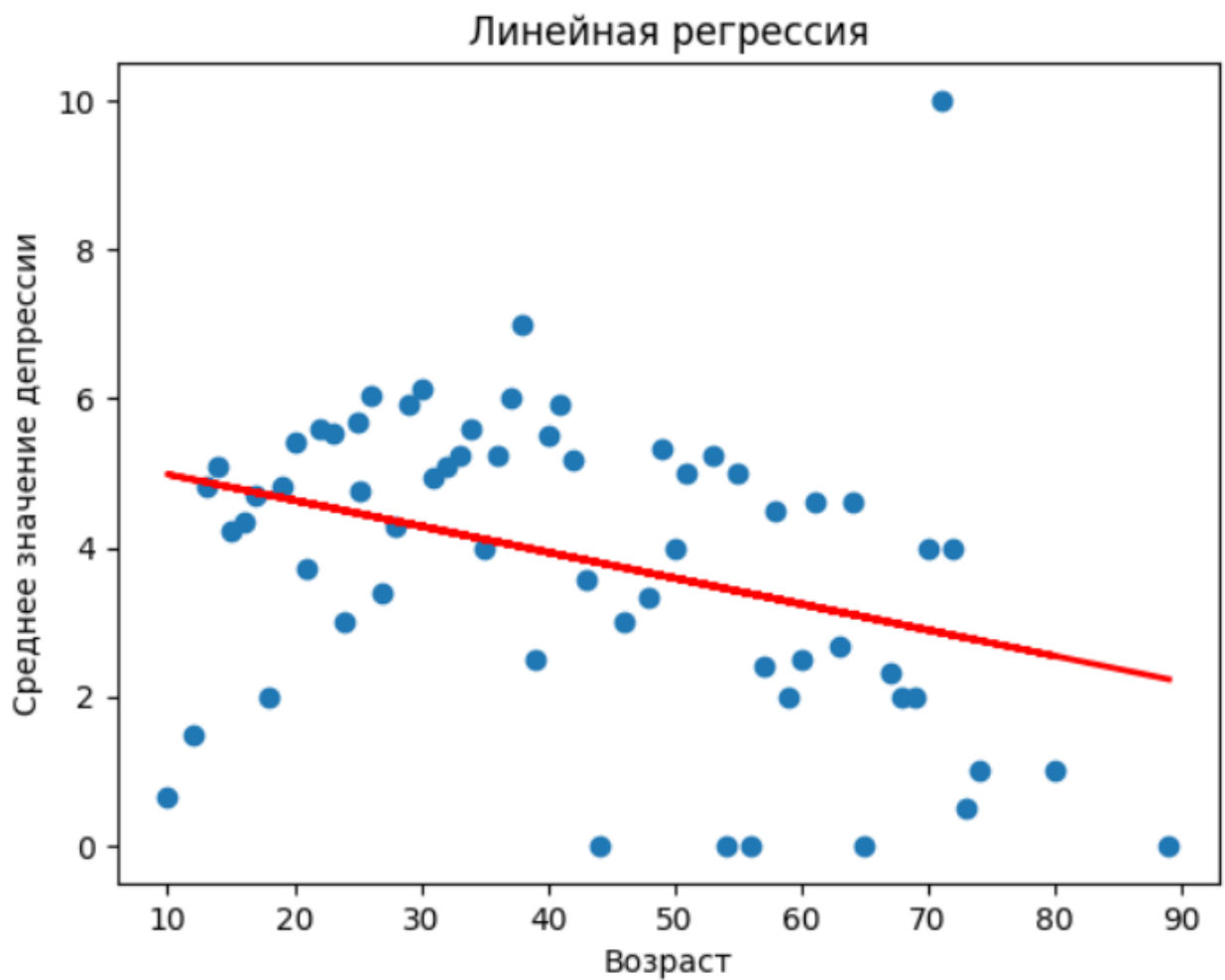


Рисунок 4 – Линейная регрессия в Python

R (lm())

Модель регрессии в R построена с использованием функции `lm()`. Код представлен в Листинге 4.

Листинг 4 – Построение регрессии в R

```
df$Depression[is.na(df$Depression)] <- mean(df$Depression, na.rm = TRUE)
df$Age[is.na(df$Age)] <- mean(df$Age, na.rm = TRUE)
Y <- aggregate(Depression ~ Age, data = df, FUN = mean)
# Создание модели линейной регрессии
model <- lm(Depression ~ Age, data = Y)
summary(model) # Вывод сводной статистики
# Построение графика
plot(Y$Age, Y$Depression,
     main = "Линейная регрессия",
     xlab = "Возраст",
     ylab = "Среднее значение депрессии",
     col = "blue",
     pch = 19)
# Добавление линии регрессии
abline(model, col = "red", lwd = 2)
```

Результаты представлены на Рисунке 5.

```
Call:
lm(formula = Depression ~ Age, data = Y)

Residuals:
    Min       1Q   Median       3Q      Max
-3.7044 -1.3078  0.2407  1.1940  7.2096

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  5.61540    0.56029  10.022 1.96e-14 ***
Age         -0.04154    0.01191  -3.489 0.000916 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.872 on 60 degrees of freedom
Multiple R-squared:  0.1686,    Adjusted R-squared:  0.1548
F-statistic: 12.17 on 1 and 60 DF,  p-value: 0.000916
```

Рисунок 5 – Результаты линейной регрессии в R
Визуализация модели на языке R представлена на рис. 6.

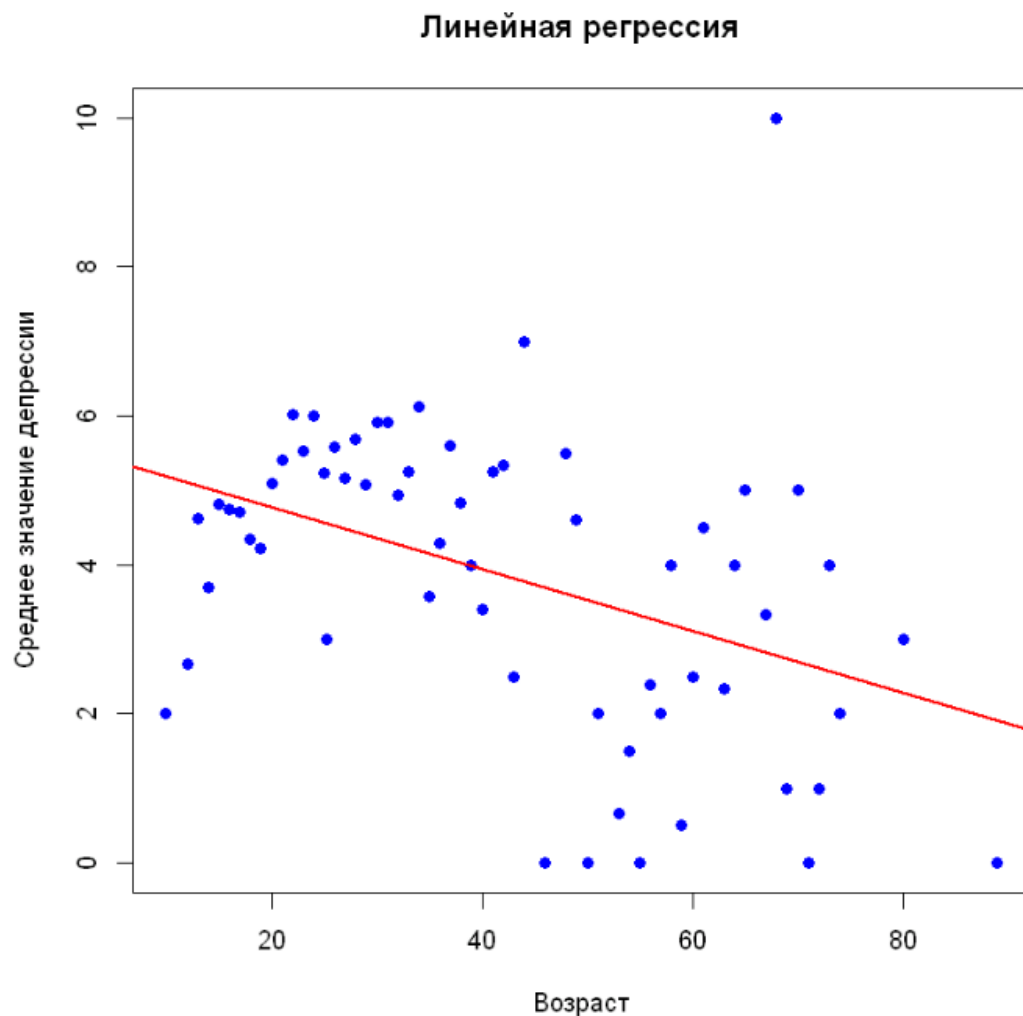


Рисунок 6 – Линейная регрессия в R

Сравнение результатов между инструментами

Сравнение корреляционного анализа представлено в табл. 1.

Переменная 1	Переменная 2	Корреляция Пирсона (Python)	Корреляция Пирсона (R)	Корреляция Спирмана (Python)	Корреляция Спирмана (R)
Age	Hours per day	-0.0446	-0.0446	-0.1409	-0.1406
	BPM	-0.0299	-0.0268	0.0057	0.0657
	Anxiety	-0.1767	-0.1765	-0.0694	-0.0682
	Depression	-0.1216	-0.1215	0.0089	0.0083
	Insomnia	0.0069	0.0069	0.0216	0.0212
	OCD	-0.1301	-0.1299	-0.0912	-0.0898
Hours per day	BPM	0.0426	0.0401	0.0118	-0.0713
	Anxiety	0.0493	0.0493	0.0943	0.0943
	Depression	0.1105	0.1105	0.1352	0.1352
	Insomnia	0.1418	0.1418	0.1481	0.1481
	OCD	0.1187	0.1187	0.1254	0.1254
BPM	Anxiety	-0.0271	-0.0248	0.0312	0.0071
	Depression	0.0414	0.0379	0.0519	-0.0038
	Insomnia	0.0536	0.0500	0.0618	0.0112
	OCD	0.0189	0.0176	-0.0231	-0.0211
Anxiety	Depression	0.5199	0.5200	0.5117	0.5117
	Insomnia	0.2927	0.2927	0.2938	0.2938
	OCD	0.3483	0.3483	0.3429	0.3429
Depression	Insomnia	0.3790	0.3790	0.3863	0.3863
	OCD	0.1970	0.1970	0.2082	0.2082
Insomnia	OCD	0.2264	0.2264	0.2407	0.2407

Таким образом, значения корреляций на разных языках практически совпадают – полностью вплоть до 2ого знака после запятой.

Коэффициенты линейной регрессии в Python и R различаются незначительно, но коэффициент детерминации в R выше (0.1548 против 0.104 для adjusted). В обоих случаях p-value очень близок к нулю.

ЗАКЛЮЧЕНИЕ

Python — мощная экосистема для анализа и машинного обучения, позволяет детально настраивать модели, визуализацию и использовать множество библиотек для расширенных методов.

R — классический язык статистики, богатая коллекция пакетов и удобные функции для быстрой проверки гипотез и визуализации.

В рамках практической работы были изучены и сравнены методы корреляционного анализа и построения линейных регрессионных моделей с использованием языков программирования Python и R. Основные выводы:

1. Сравнение корреляционных коэффициентов:
 - a. Коэффициенты корреляции Пирсона и Спирмена, рассчитанные в Python и R, практически полностью совпадают (до 2–3 знаков после запятой). Это подтверждает надёжность обоих инструментов для анализа взаимосвязей между переменными.
 - b. Выявлено, что наиболее значимые корреляции наблюдаются между переменными **Anxiety** и **Depression** (Пирсон: ~ 0.52 , Спирмен: ~ 0.51), а также между **Depression** и **Insomnia** (Пирсон: ~ 0.38 , Спирмен: ~ 0.39).
2. Регрессионный анализ:
 - a. Результаты линейной регрессии показали незначительные различия между Python и R. Коэффициенты наклона и свободного члена практически идентичны, что говорит о согласованности расчётов.
 - b. Значение коэффициента детерминации (R^2) в R выше (adjusted $R^2=0.1548$ против 0.104 в Python), что может быть связано с особенностями обработки данных (например, порядком сортировки или методами обработки пропусков).
 - c. Р-значения для коэффициентов модели в обоих случаях оказались крайне малыми (<0.01), что указывает на статистическую значимость полученных результатов.
3. Преимущества Python:
 - a. Гибкость и возможность использования широкого спектра библиотек для анализа данных (**pandas**, **statsmodels**, **seaborn**, **matplotlib**) и машинного обучения.
 - b. Детальная настройка моделей и визуализации, что особенно полезно для сложных проектов.
4. Преимущества R:
 - a. Быстрое выполнение стандартных статистических процедур благодаря

встроенным функциям (**cor()**, **lm()**) и специализированным пакетам (**ggcorrplot**, **broom**).

- b. Удобство интерпретации результатов через компактные отчёты и графики.

5. Ограничения:

- a. Python требует более подробного кодирования даже для базовых задач, что может быть затратным для начинающих пользователей.
- b. R менее удобен для масштабных проектов или интеграции с современными технологиями машинного обучения.

Таким образом, выбор между Python и R зависит от целей исследования и уровня подготовки пользователя. Для быстрого анализа данных и проверки гипотез предпочтительнее использовать R, а для комплексного анализа и разработки моделей машинного обучения — Python. Оба инструмента демонстрируют высокую точность и согласованность результатов при правильной подготовке данных.